

DOCUMENTO TÉCNICO DE DISEÑO

DOCUMENTO TÉCNICO DE IMPLEMENTACIÓN OPTIMIZACIÓN DE RENDIMIENTO EN MONGODB - ETAPA UNO

DISEÑO Y OPTIMIZACIÓN DE BASES DE DATOS

EQUIPO E21

NESTOR ALEJANDRO RODRIGUEZ BENAVIDES - NESTORROBE@UNISABANA.EDU.CO

CARLOS DANIEL SANDOVAL - CARLOSSANDPAR@UNISABANA.EDU.CO

PETER ALEXANDER PALACIOS GARNICA - PETERPAGA@UNISABANA.EDU.CO

JUAN GUILLERMO OSSA SÁNCHEZ - JUANOSSA@UNISABANA.EDU.CO

MAESTRÍA EN ARQUITECTURA DE SOFTWARE

DOCENTE: MIGUEL ALFONSO VARELA

14 DE JUNIO DE 2026

Índice

1. Resumen ejecutivo.....	4
2. Alcance y continuidad con Ecommify.....	4
3. Metodología y trazabilidad del Colab.....	4
4. Entorno de trabajo y colecciones.....	5
5. Consultas críticas Q01-Q07.....	5
6. Línea base de rendimiento con explain.....	6
7. Índices implementados o reutilizados.....	6
8. Comparación antes/después.....	7
9. Optimización del aggregation pipeline.....	8
10. Sharding y replica sets teórico/simulado.....	9
11. Monitoreo, limitaciones y acciones de cierre.....	12
12. Conclusiones.....	13
13. Referencias.....	13
Anexo A. Mapa de evidencias Colab-Word.....	13
Anexo B. Resumen técnico consolidado.....	14

1. Resumen ejecutivo

El trabajo se concentró en las colecciones `product_catalog` y `product_reviews`, manteniendo la arquitectura definida en entregas anteriores: PostgreSQL como núcleo transaccional y MongoDB como componente documental para catálogo enriquecido, reseñas y análisis flexible.

La implementación se organizó en ocho hitos dentro de Google Colab. Se definieron 7 consultas críticas, se capturó una línea base con `explain("executionStats")`, se registraron 5 índices creados y 2 índices reutilizados, se compararon métricas antes/después, se optimizó un aggregation pipeline y se realizó una simulación de sharding con datos reales del catálogo.

El hallazgo más importante en la medición de consultas fue Q04: la búsqueda textual pasó de COLLSCAN a IXSCAN/TEXT_MATCH mediante el índice de texto existente `idx_text_product_search`. Q07 redujo el tiempo de ejecución de 4 ms a 3 ms. En las demás consultas, el impacto en tiempo fue bajo o variable porque el dataset académico ya contaba con índices previos y porque varias consultas operan sobre volúmenes pequeños.

En el análisis de distribución para sharding se evaluaron 971 documentos de `product_catalog`. La categoría con mayor participación fue `bed_bath_table`, con 8.86% del total; el HHI calculado fue 0.0483, lo que sugiere una distribución poco concentrada. La simulación de shards mostró que la estrategia `category + hashed product_id` tuvo el menor desbalance (0.0227), por lo que fue seleccionada como estrategia teórica principal para `product_catalog`.

Evidencia: notebook de Colab, sección Hito 8.1, Hito 8.3 y Hito 8.4.

2. Alcance y continuidad con Ecommify

El alcance de esta etapa se limita a la optimización MongoDB solicitada para la Unidad 5. No se modificó el diseño transaccional en PostgreSQL, no se recargaron datos y no se habilitó sharding real. La actividad mantiene continuidad con el diseño híbrido de Ecommify: las operaciones transaccionales críticas permanecen en PostgreSQL, mientras que MongoDB se usa para consultas de catálogo, reseñas y análisis documental.

La etapa no reemplaza los documentos previos. En cambio, toma como base el diseño documental de la Unidad 3 y la optimización PostgreSQL de la Unidad 4 para cerrar el componente MongoDB de rendimiento.

Evidencia: notebook de Colab, sección Hito 1 y Hito 8.4.

3. Metodología y trazabilidad del Colab

Esta actividad, exige optimizaciones en MongoDB mediante índices compuestos, parciales y de texto; validación con `explain("executionStats")`; optimización de aggregation pipeline; diseño teórico de sharding y replica sets; y monitoreo de rendimiento. Para mantener orden y reproducibilidad, el notebook se estructuró por hitos.

Evidencia: notebook de Colab, sección Hito 8.11 y Hito 8.2.

Tabla 1. Estado final de hitos ejecutados en Colab.

Hito	Nombre	Estado	Evidencia principal
Hito 1	Conexión y reconocimiento de colecciones	Completado	Conexión exitosa, conteos e índices iniciales.
Hito 2	Definición de consultas críticas	Completado	Q01-Q07 definidas y validadas funcionalmente.
Hito 3	Medición base con explain	Completado	baseline_df y CSV de línea base.
Hito 4	Creación y reutilización de índices	Completado	Índices creados/reutilizados, inventario final de índices.
Hito 5	Comparación antes/después	Completado	comparison_df con mejoras por consulta.
Hito 6	Optimización de aggregation pipeline	Completado	Pipeline base vs optimizado y tabla de stages.
Hito 7	Sharding y replica sets teórico/simulado	Completado	Simulación de shards, decisión de shard keys y estrategia de réplica.
Hito 8	Cierre de evidencias	Completado	Mapa de evidencias para el documento Word.

4. Entorno de trabajo y colecciones

El entorno validado corresponde a MongoDB Atlas, conectado desde Google Colab mediante PyMongo. En la base ecommify_mongodb se trabajó con las colecciones product_catalog y product_reviews. product_catalog representa el catálogo enriquecido de productos; product_reviews almacena reseñas como colección referenciada para análisis de satisfacción y reputación.

Evidencia: notebook de Colab, sección Hito 1.

5. Consultas críticas Q01-Q07

Se definieron siete consultas críticas para cubrir navegación de catálogo, filtros por categoría/precio, consulta por vendedor, búsqueda textual, reseñas por producto, reseñas críticas y análisis por categoría. Estas consultas fueron la base para la línea base, la creación/reutilización de índices y la comparación posterior.

Tabla 2. Consultas críticas definidas para la optimización MongoDB.

Consulta	Colección	Operación	Propósito de negocio
Q01	product_catalog	find	Productos por categoría ordenados por relevancia comercial/ventas.
Q02	product_catalog	find	Filtro por categoría y precio mínimo, orientado a regla ESR.
Q03	product_catalog	find	Productos asociados a un vendedor.
Q04	product_catalog	find	Búsqueda textual por nombre, palabras clave y categoría.
Q05	product_reviews	find	Reseñas asociadas a un producto.
Q06	product_reviews	find	Reseñas de baja calificación para análisis de satisfacción.
Q07	product_catalog	aggregate	Distribución/analítica por categoría.

Evidencia: notebook de Colab, sección Hito 2.5 a Hito 2.7.

6. Línea base de rendimiento con explain

La línea base se obtuvo mediante `explain("executionStats")` para cada consulta crítica. Las métricas principales fueron `execution_time_ms`, `total_docs_examined`, `total_keys_examined`, `n_returned`, índices utilizados y clasificación del plan. Esta medición representa el estado antes de las optimizaciones de esta etapa; no necesariamente equivale a un estado sin índices, porque Atlas ya contenía algunos índices previos.

Tabla 3. Línea base de rendimiento por consulta crítica.

Q	Colección	Operación	Tiempo ms	Docs exam.	Keys exam.	Retornados	Plan
Q01	product_catalog	find	1	2	2	2	IXSCAN
Q02	product_catalog	find	0	1	1	1	IXSCAN
Q03	product_catalog	find	0	1	1	1	IXSCAN
Q04	product_catalog	find	4	1000	0	1000	COLLSCAN
Q05	product_reviews	find	0	1	1	1	IXSCAN
Q06	product_reviews	find	42	15275	15275	15275	IXSCAN
Q07	product_catalog	aggregate	4	0	972	971	IXSCAN

Evidencia: notebook de Colab, Hito 3.2 a Hito 3.6. Archivo: `hito3_baseline_explain_summary.csv`.

En la línea base, la mayoría de consultas ya usaban IXSCAN. La excepción más clara fue Q04, que inició con COLLSCAN y examinó 1000 documentos sin llaves examinadas. Esta consulta se convirtió en el principal caso para validar búsqueda full-text.

7. Índices implementados o reutilizados

La etapa de indexación creó índices compuestos, índices orientados a regla ESR, un índice parcial para reseñas de baja calificación y reutilizó índices existentes cuando MongoDB detectó equivalencia. Esta decisión evitó sobre indexación y conflictos de opciones en Atlas.

Tabla 4. Índices creados o reutilizados durante el Hito 4.

Colección	Índice	Tipo	Estado	Propósito
product_catalog	idx_catalog_category_sales	compound	OK	Optimizar Q01: productos por categoría ordenados por ventas.
product_catalog	idx_catalog_category_price	compound_esr	OK	Optimizar Q02: filtro por categoría y rango/orden de precio.
product_catalog	idx_catalog_seller_name	compound	OK	Optimizar Q03: consulta de productos asociados a un vendedor.
product_catalog	idx_text_product_search	text_existing	REUSED	Reutilizar índice de texto existente para Q04: búsqueda textual en nombre, palabras clave y categoría.
product_reviews	idx_reviews_product_date	compound	OK	Optimizar Q05: consulta de reseñas por producto ordenadas por fecha.
product_reviews	idx_reviews_low_score_partial	partial	OK	Optimizar Q06: reseñas con baja calificación.
product_catalog	idx_category_name_translated	single_field_existing	REUSED	Reutilizar índice existente para Q07: agregación y análisis por categoría.

Evidencia: notebook de Colab, Hito 4.3 a Hito 4.12. Archivos: `hito4_created_indexes.csv` y `hito4_index_inventory.csv`.

Dos decisiones merecen registro especial: el índice de texto `idx_text_product_search` ya existía y cubría `category.name_translated`, `name` y `search_keywords`; por eso se utilizó para Q04. Asimismo, el índice `idx_category_name_translated` ya existía y se utilizó para análisis por categoría en Q07.

8. Comparación antes/después

Después de la indexación se ejecutaron nuevamente las consultas críticas. La lectura de resultados debe considerar que varias consultas ya estaban optimizadas antes de esta etapa y que el volumen académico puede generar variaciones de milisegundos no concluyentes.

Tabla 5. Comparación cuantitativa antes/después.

Q	Tiempo antes	Tiempo después	Docs antes	Docs después	Keys antes	Keys después	Plan antes	Plan después
Q01	1	5	2	2	2	2	IXSCAN	IXSCAN
Q02	0	0	1	1	1	1	IXSCAN	IXSCAN
Q03	0	3	1	1	1	1	IXSCAN	IXSCAN
Q04	4	20	1000	1000	0	1000	COLLSCAN	IXSCAN
Q05	0	6	1	1	1	1	IXSCAN	IXSCAN
Q06	42	106	15275	15275	15275	15275	IXSCAN	IXSCAN
Q07	4	3	0	0	972	972	IXSCAN	IXSCAN

Evidencia: notebook de Colab, Hito 5.6 a Hito 5.8. Archivo: `hito5_before_after_comparison.csv`.

Figura 1. Comparación de tiempo de ejecución antes/después.

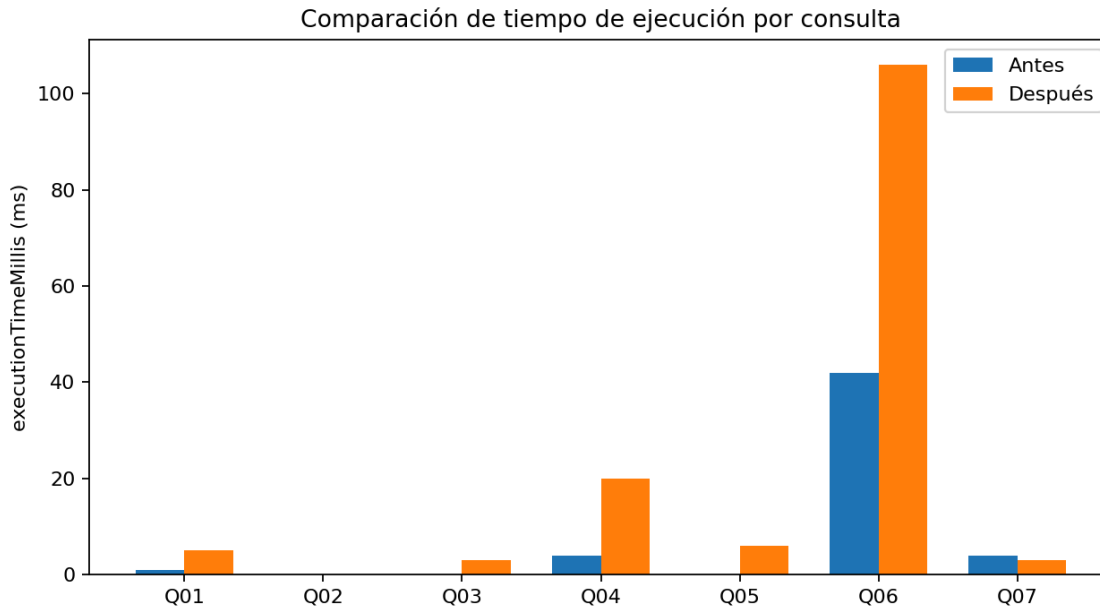


Figura 2. Documentos examinados por consulta, escala logarítmica.

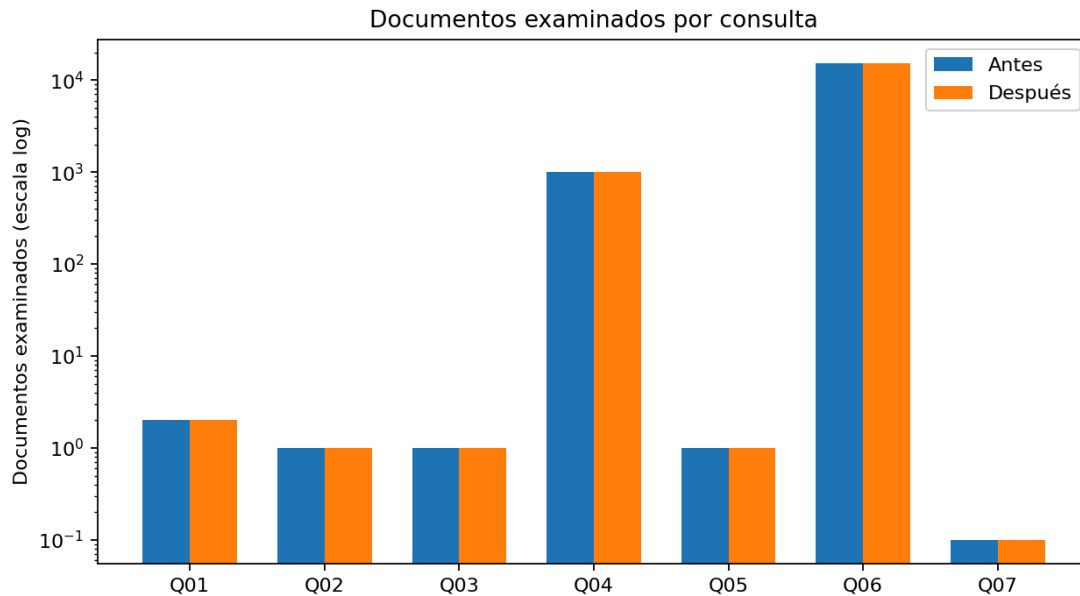


Tabla 6. Interpretación técnica por consulta.

Q	Índices posteriores	Interpretación técnica
Q01	idx_catalog_category_price, idx_catalog_category_sales...	No se observa mejora significativa o la consulta ya estaba optimizada.
Q02	idx_catalog_category_price, idx_catalog_category_sales...	No se observa mejora significativa o la consulta ya estaba optimizada.
Q03	idx_catalog_seller_name, idx_seller_summary_seller_id	No se observa mejora significativa o la consulta ya estaba optimizada.
Q04	idx_text_product_search	Cambió de escaneo completo a uso de índice.
Q05	idx_reviews_product_date, idx_reviews_product_id...	No se observa mejora significativa o la consulta ya estaba optimizada.
Q06	idx_reviews_low_score_partial, idx_reviews_score	No se observa mejora significativa o la consulta ya estaba optimizada.
Q07	idx_catalog_category_price, idx_catalog_category_sales...	Redujo tiempo de ejecución.

La mejora cualitativa más importante fue Q04: pasó de revisión completa de colección a uso del índice de texto. Q07 redujo el tiempo de ejecución de 4 ms a 3 ms. Q01, Q02, Q03, Q05 y Q06 conservaron IXSCAN antes y después, por lo que no se observó una mejora significativa en documentos examinados.

9. Optimización del aggregation pipeline

Se construyó un pipeline analítico para responder cuáles categorías presentan mejor desempeño comercial y reputacional considerando catálogo, ventas y reseñas. La versión optimizada aplicó \$match y \$project temprano antes de \$lookup, con el propósito de reducir el volumen de documentos y campos procesados.

Tabla 7. Stages del pipeline optimizado.

Orden	Stage	Propósito
1	\$match	Filtrar temprano por categoría para reducir documentos procesados.
2	\$project	Reducir campos antes de hacer lookup y agrupaciones.
3	\$lookup	Unir productos del catálogo con sus reseñas.
4	\$unwind	Descomponer el arreglo de reseñas para calcular métricas.
5	\$group	Agrupar por categoría y calcular métricas agregadas.
6	\$project	Formatear la salida final del análisis.
7	\$sort	Ordenar categorías por desempeño comercial.

Evidencia: notebook de Colab, Hito 6.2 a Hito 6.10. Archivos: hito6_pipeline_comparison.csv, hito6_pipeline_improvement.csv y hito6_pipeline_stages.csv

Tabla 8. Comparación de pipeline base vs optimizado.

Versión	Tiempo ms	Docs exam.	Keys exam.	Retornados
BASE	1	3	3	2
OPTIMIZED	1	3	3	2

El pipeline base y el optimizado obtuvieron 1 ms de tiempo de ejecución, 3 documentos examinados, 3 llaves examinadas y 2 resultados retornados. La mejora porcentual fue 0% porque la consulta ya estaba muy acotada y apoyada por índices. Aun así, la versión optimizada conserva una estructura más recomendable: filtra temprano, proyecta campos necesarios y luego ejecuta el join con reseñas.

10. Sharding y replica sets teórico/simulado

El Hito 7 combinó análisis práctico local y diseño teórico. Las secciones 7.2 a 7.7 analizaron distribución y simularon shards con datos reales; las secciones 7.8 a 7.11 generaron tablas documentales con comandos y decisiones teóricas, sin modificar Atlas.

Tabla 9. Top 10 categorías por volumen en product_catalog.

Categoría	Documentos	Participación
bed bath table	86	8.86%
sports leisure	83	8.55%
health beauty	75	7.72%
furniture decor	72	7.42%
housewares	64	6.59%
auto	57	5.87%
computers accessories	45	4.63%
toys	45	4.63%
telephony	38	3.91%
watches gifts	36	3.71%

Evidencia: notebook de Colab, Hito 7.2 a Hito 7.12. Archivos: hito7_category_distribution.csv, hito7_shard_strategy_metrics.csv, hito7_decision_summary.csv.

Figura 3. Top 10 categorías por número de productos.

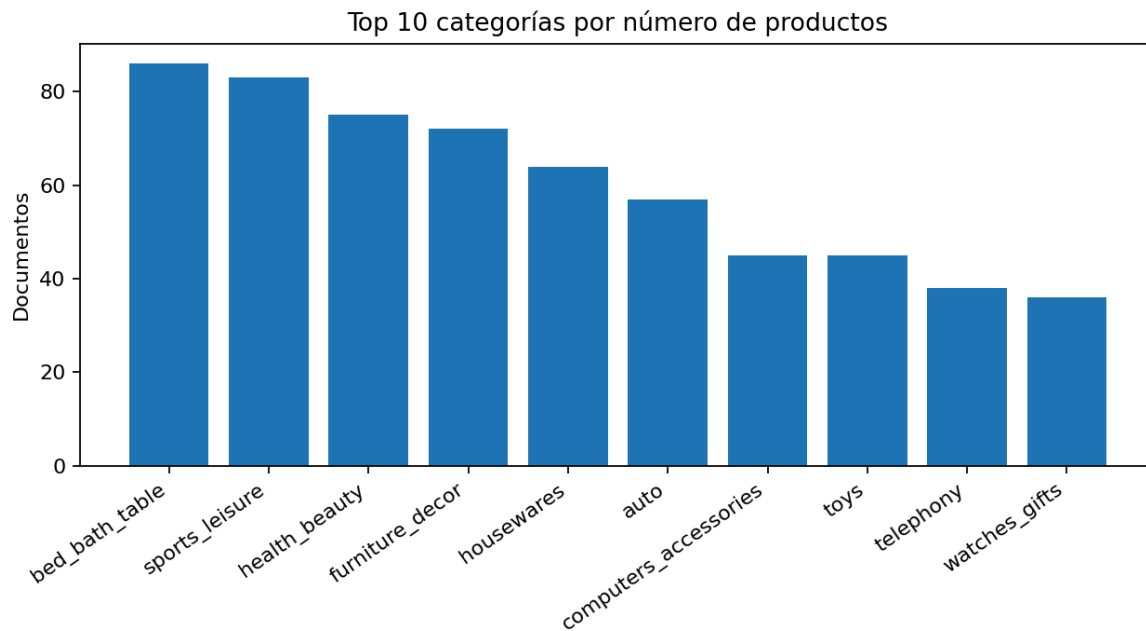


Tabla 10. Métricas de concentración de categorías.

Métrica	Valor
total_documents_analyzed	971.0000
max_category_share	0.0886
hhi_category_concentration	0.0483

El HHI de 0.0483 y una participación máxima de 8.86% indican baja concentración. Sin embargo, la simulación por categoría generó mayor desbalance entre shards que las estrategias con product_id hashed.

Tabla 11. Comparación de estrategias simuladas de shard key.

Estrategia	Docs	Max share	Min share	Desbalance
shard_by_category	971	0.4511	0.2503	0.2008
shard_by_product_hash	971	0.3450	0.3193	0.0257
shard_by_category_product	971	0.3460	0.3234	0.0227

Figura 4. Desbalance simulado por estrategia de shard key.

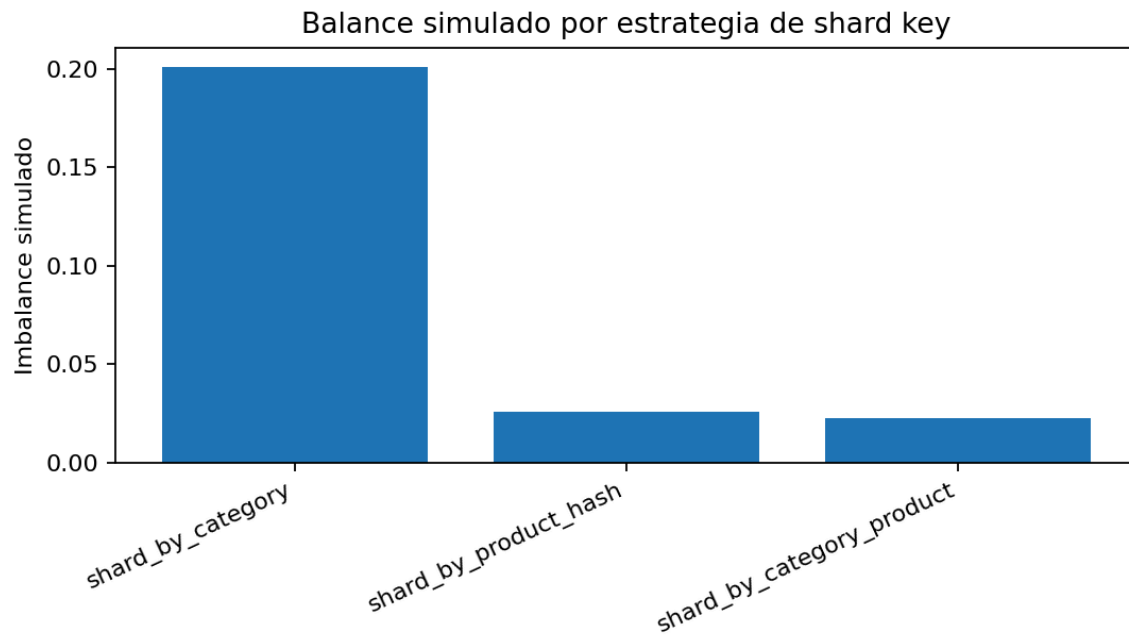


Tabla 12. Matriz de decisión de shard keys.

Colección	Shard key candidata	Ventajas	Riesgos	Decisión
product_catalog	category	Ayuda a consultas por categoría.	Puede generar hotspots si pocas categorías concentran muchos productos.	No seleccionada como única clave.
product_catalog	hashed product_id	Distribuye mejor los documentos.	Las consultas por categoría podrían requerir búsqueda en varios shards.	Candidata para balance, pero limitada para navegación por categoría.
product_catalog	category + hashed product_id	Combina localidad por categoría con mejor distribución por producto.	Mayor complejidad de diseño e índices compatibles.	Seleccionada como estrategia teórica principal.
product_reviews	hashed product_id	Distribuye reseñas y favorece consultas por producto.	Agregaciones globales pueden requerir fan-out.	Seleccionada como estrategia teórica para reseñas.

Tabla 13. Resumen de decisiones de sharding y réplica.

Tema	Decisión	Razón
Shard key product_catalog	category + hashed product_id	Equilibra consultas por categoría con mejor distribución de productos.
Shard key product_reviews	hashed product_id	Las reseñas se consultan principalmente por producto.
Replica set	1 primary + 2 secondaries	Permite alta disponibilidad y lecturas no críticas desde réplicas.
Write concern	majority para cargas y actualizaciones	Aumenta garantía de durabilidad en el módulo documental.
Read preference	secondaryPreferred para analítica; primaryPreferred para datos recientes	Balancea rendimiento y consistencia según el tipo de consulta.

Tabla 14. Estrategia teórica de replica set.

Componente	Rol	Uso en Ecommify
Primary node	Recibe escrituras y operaciones críticas.	Actualización de catálogo, carga de reseñas y cambios controlados.
Secondary node 1	Replica datos y puede atender lecturas no críticas.	Consultas analíticas o de catálogo con tolerancia a ligera latencia.
Secondary node 2	Aumenta disponibilidad y soporta failover.	Continuidad operativa ante falla del primary.

Tabla 15. Estrategia de lectura/escritura por operación.

Tipo operación	Consultas	Read preference	Read concern	Write concern	Justificación
Lectura de catálogo	Q01, Q02, Q03, Q04	secondaryPreferred	local	No aplica	Son consultas de lectura donde una pequeña latencia de réplica puede ser aceptable.
Lectura de reseñas por producto	Q05	primaryPreferred	majority	No aplica	Las reseñas recientes pueden requerir mayor consistencia.
Análisis de reseñas críticas	Q06	secondaryPreferred	local	No aplica	Consulta analítica tolerante a ligera consistencia eventual.
Carga o actualización documental	Insertión/actualización de product_catalog y product_reviews	No aplica	No aplica	majority	Se prioriza durabilidad y confirmación de escritura replicada.

11. Monitoreo, limitaciones y acciones de cierre

El monitoreo implementado en el Colab se basó en explain("executionStats"), clasificación de planes, métricas de documentos/llaves examinadas, inventario de índices y comparación antes/después. Estos elementos permiten evaluar el impacto técnico sin depender únicamente del tiempo de ejecución.

Tabla 16. Limitaciones y decisiones de alcance.

Limitación o decisión	Justificación
No se habilitó sharding real.	La etapa requiere análisis/simulación y el entorno Atlas gratuito no está orientado a cambios administrativos avanzados de sharding.
No se modificó el diseño transaccional en PostgreSQL.	La Unidad 5 se enfoca en optimización MongoDB para el módulo analítico/documental de Ecommify.
No se recargaron datos.	Las colecciones product_catalog y product_reviews ya estaban disponibles desde la continuidad del proyecto.
Se utilizaron índices existentes cuando fue posible.	Se evitó sobreindexación y conflictos de índices equivalentes en MongoDB Atlas.
Las métricas dependen del tamaño actual de las colecciones.	En colecciones pequeñas, la mejora en tiempo puede ser baja; por eso también se analizaron planes, documentos examinados e índices usados.

Evidencia: notebook de Colab, Hito 8.4 y Hito 8.5.

12. Conclusiones

- Las consultas Q01-Q07 permitieron cubrir navegación de catálogo, búsqueda textual, reseñas y analítica por categoría.
- La indexación fue conservadora: se crearon índices necesarios y se reutilizaron índices existentes para evitar sobreindexación.
- Q04 evidencia la mejora más clara de plan de ejecución: pasó de COLLSCAN a uso de índice de texto.
- El pipeline optimizado no mejoró métricas por tamaño/acotamiento del dataset, pero quedó estructurado con buenas prácticas de \$match y \$project tempranos.
- La estrategia teórica seleccionada para product_catalog fue category + hashed product_id, porque combinó soporte para consultas por categoría con mejor distribución simulada.
- La estrategia de réplica propuesta diferencia lecturas analíticas tolerantes a latencia y escrituras con write concern majority.

13. Referencias

- Fuente 1. MongoDB, Inc. (2025). MongoDB performance best practices. MongoDB Manual. <https://www.mongodb.com/es/docs/manual/administration/analyzing-mongodb-performance/>
- Fuente 2. Bradshaw, S., Brazil, E., & Chodorow, K. (2019). MongoDB: The definitive guide: Powerful and scalable data storage (3rd ed.). O'Reilly Media.
- Fuente 3. Borucki, A. (2025). MongoDB 8.0 in action (3rd ed.). Manning Publications.
- Universidad de La Sabana. (2026). Guía de actividades Unidad 5: Optimización de rendimiento en MongoDB.
- Equipo Ecommify. (2026). Documento técnico de diseño: Diseño conceptual y lógico de la base de datos del proyecto Ecommify.
- Equipo Ecommify. (2026). Notebook de Google Colab: Optimización MongoDB - Unidad 5, Etapa 1. [URL pendiente de anexar por el equipo].

Anexo A. Mapa de evidencias Colab-Word

Tabla A1. Mapa de evidencias usado para construir el documento.

Sección Word	Referencia Colab	Evidencia	Uso
1. Resumen ejecutivo	Hito 8.1	Estado general de los hitos ejecutados.	Explicar alcance y resultado global de la etapa.
2. Entorno de trabajo y colecciones	Hito 1	Conexión a MongoDB Atlas, colecciones product_catalog y product_reviews.	Demostrar que el módulo documental ya estaba cargado y disponible.
3. Consultas críticas	Hito 2.5 - Hito 2.7	Consultas Q01-Q07 y validación funcional.	Justificar los escenarios de optimización.
4. Línea base de rendimiento	Hito 3.2 - Hito 3.6	baseline_df, baseline_efficiency_df y CSV de línea base.	Presentar métricas antes de optimizar.
5. Índices implementados o reutilizados	Hito 4.3 - Hito 4.12	created_indexes_df e index_inventory_df.	Explicar índices compuestos, parciales y de texto.
6. Comparación antes/después	Hito 5.6 - Hito 5.8	comparison_df y archivos hito5.	Mostrar impacto de optimización con métricas.
7. Optimización del aggregation pipeline	Hito 6.2 - Hito 6.10	pipeline_comparison_df, pipeline_improvement_df y pipeline_stages_df.	Documentar proceso de optimización del pipeline.
8. Sharding y replica sets	Hito 7.2 - Hito 7.12	Distribución por categoría, simulación de shards y estrategia teórica.	Justificar shard keys, replica set, read/write concerns.

9. Conclusiones y limitaciones	Hito 8.3 - Hito 8.5	Resumen técnico, limitaciones y archivos exportados.	Cerrar el documento con hallazgos y alcance real.
--------------------------------	---------------------	--	---

Anexo B. Resumen técnico consolidado

Tabla B1. Resumen técnico exportado del Hito 8.

Tema	Resultado
Consultas críticas	Se definieron 7 consultas críticas para catálogo, búsqueda textual, reseñas y analítica.
Índices	Se registraron 5 índices creados y 2 índices reutilizados para evitar sobreindexación.
Comparación antes/después	Se compararon métricas de executionTimeMillis, documentos examinados, llaves examinadas y planes de ejecución.
Aggregation pipeline	Se construyó un pipeline base y una versión optimizada con \$match y \$project tempranos.
Sharding y replica sets	Se realizó simulación de distribución y se propuso estrategia teórica de shard keys, réplica y concerns.